

FIAP — Global Solution | 1º Semestre de 2026 | Dynamic Programming

Monitoramento de Riscos Ambientais com Árvores, Grafos e Algoritmos

Força Bruta · Algoritmos Gulosos · Estruturas de Dados

1. Contextualização

1.1 O cenário

O Brasil é um dos países mais vulneráveis ao impacto das mudanças climáticas. Eventos como a seca histórica no Rio Amazonas em 2023, as enchentes no Rio Grande do Sul em 2024 e a expansão do desmatamento na Amazônia impõem desafios urgentes de monitoramento, planejamento e resposta rápida. Satélites como os da constelação Sentinel (ESA) e o GOES-16 transmitem dados em tempo real sobre temperatura, precipitação e cobertura vegetal — gerando redes complexas de informação geoespacial que precisam ser organizadas, percorridas e analisadas com eficiência.

Para resolver esses problemas, engenheiros de software precisam dominar as ferramentas fundamentais da computação: estruturas de dados que organizam a informação, algoritmos que percorrem grafos e árvores, e estratégias de busca que equilibram qualidade e desempenho.

Conexão Global Solution 2026: este projeto traduz o ecossistema da economia espacial em um problema computacional concreto, usando dados de satélites para construir sistemas de monitoramento, via códigos, dos ambientes no Brasil, alinhando-se aos ODS 2, 9, 11 e 13 da ONU.

1.2 O sistema a construir

Seu grupo foi contratado por um consórcio de defesa civil e cooperativas agrícolas para desenvolver um sistema de monitoramento e triagem de riscos ambientais em municípios brasileiros. O sistema deve:

- Representar os municípios e suas conexões (estradas, hidrovias, rotas de suprimento) como um grafo ponderado;

- Organizar os dados de risco de cada município em uma árvore binária de busca (BST), permitindo consultas eficientes por índice de risco;
- Usar listas, tuplas, dicionários e filas para armazenar, transferir e processar os atributos de cada nó e aresta;
- Implementar Força Bruta para encontrar a solução ótima em instâncias pequenas e validar o algoritmo Guloso;
- Implementar um algoritmo Guloso para resolver instâncias reais de forma eficiente;
- Comparar os dois algoritmos em qualidade de solução, tempo de execução e uso de memória.

2. Modelagem das Estruturas de Dados

2.1 Grafo de municípios

O grafo $G = (V, E)$ representa a rede de municípios de uma região brasileira escolhida pelo grupo. Cada vértice $v \in V$ é um município, armazenado como uma tupla:

```
# Cada vértice: tupla imutável com atributos principais
vertice = (id_municipio, nome, indice_risco, custo_atendimento,
populacao)

# Exemplo – município do RS:
v = (4314902, 'Porto Alegre', 0.72, 1850.0, 1400000)
```

Cada aresta $e = (u, v, \text{peso}) \in E$ representa uma rota entre municípios, onde o peso equivale ao tempo de deslocamento (horas) ou distância (km). O grafo deve ser implementado como dicionário de listas de adjacência:

```
# Dicionário de adjacência: {id_vertice: [(vizinho, peso), ...]}
grafo = {
    4314902: [(4300406, 2.5), (4316808, 1.8)],
    4300406: [(4314902, 2.5), (4307005, 3.1)],
    # ...
}
```

O grupo deve justificar no relatório a escolha desta representação (lista de adjacência vs. matriz de adjacência) em termos de complexidade de espaço e tempo para as operações realizadas.

2.2 Árvore Binária de Busca (BST) de municípios por risco

Uma BST é construída sobre os vértices do grafo, usando o índice de risco r como chave de ordenação. A árvore deve suportar as seguintes operações, implementadas pelo grupo do zero (sem bibliotecas externas):

- `inserir(nó)`: insere um município mantendo a propriedade BST ($r_{\text{esquerda}} < r_{\text{pai}} < r_{\text{direita}}$);
- `buscar(r_min, r_max)`: retorna todos os municípios com índice de risco no intervalo $[r_{\text{min}}, r_{\text{max}}]$;
- `percurso_in_order()`: retorna os municípios em ordem crescente de risco (útil para priorização);
- `altura()`: calcula a altura da árvore e avalia seu balanceamento;
- `remover(id)`: remove um nó e reequilibra ponteiros.

Exigência: a BST deve ser implementada com uma classe `Node` e uma classe `BinarySearchTree` em Python. O uso de bibliotecas como “sorted containers” ou “bintrees” não será permitido para esta seção.

2.3 Tabela de estruturas de dados utilizadas

O projeto deve fazer uso explícito e justificado das estruturas abaixo. Para cada uma, o relatório deve descrever onde e por que ela foi utilizada:

Estrutura	Uso no sistema	Aplicação concreta neste projeto
Lista (list)	Adjacência de vértices, fila de prioridade, resultado de caminho	Lista de adjacência do grafo de municípios; sequência de nós visitados no BFS/DFS
Tupla (tuple)	Par (nó, custo), coordenadas (lat, lon), aresta (u, v, peso)	Arestas do grafo satelital; nós da árvore com (id, risco, custo)
Dicionário (dict)	Mapeamento id_município → atributos, cache de subproblemas	Adjacência com pesos; metadados de nós NDVI + precipitação
Conjunto (set)	Controle de nós visitados em BFS/DFS; verificação rápida de pertencimento	Fronteira de expansão do Guloso; evitar ciclos na Força Bruta
Fila / Heap (deque / heapq)	Fila BFS; fila de prioridade para Dijkstra/Prim	heapq para extrair mínimo no Greedy; deque para BFS no grafo de risco
Árvore Binária	Hierarquia de regiões; BST de custo-benefício; árvore de decisão	Estrutura de busca binária para classificar municípios por índice de risco
Grafo (dict of lists)	Modelagem da rede de municípios e rotas de atendimento	Grafo direcionado ponderado; árvore geradora mínima (MST) de cobertura

3. Algoritmos

A solução deve considerar algoritmos abaixo:

Algoritmo	Papel no sistema	Instrução ao aluno
Força Bruta	Valida e serve de baseline para instâncias pequenas ($N \leq 12$ nós)	Enumerar todos os caminhos/subárvores; comparar com Guloso; demonstrar explosão combinatória
Guloso (Greedy)	Solução eficiente para instâncias reais; decisão local ótima a cada passo	Implementar e justificar critério de escolha local; analisar gap de otimalidade

3.1 Força Bruta — busca exaustiva no grafo

Para grafos pequenos ($N \leq 12$ vértices), implemente a enumeração completa de todos os caminhos entre a origem e o destino, ou de todas as árvores geradoras possíveis. O algoritmo deve:

- Usar recursão com ‘backtracking’ para gerar todas as possibilidades;
- Instrumentar um contador de chamadas recursivas e de caminhos avaliados;
- Registrar o custo de cada solução e identificar o ótimo global;
- Servir como oráculo de validação: para toda instância pequena, o resultado deve coincidir com o Guloso (quando o Guloso for ótimo) ou demonstrar o gap de otimização;
- Gerar gráfico do crescimento do número de caminhos em função de N , evidenciando a explosão combinatória.

3.2 Algoritmo Guloso — cobertura eficiente

Implemente uma das três variantes Greedy abaixo (apenas uma, à escolha do grupo, com justificativa):

- A. Algoritmo de Prim — construção da Árvore Geradora Mínima (MST): a cada passo, adiciona a aresta de menor peso que conecta um novo vértice à árvore já construída. Ideal para o problema de cobertura de municípios com custo mínimo de rota.
- B. Algoritmo de Kruskal — MST por ordenação de arestas: ordena todas as arestas por peso e as adiciona se não formarem ciclo (verificado com funções do tipo Union-Find). Comparar com Prim em termos de eficiência para grafos densos vs. esparsos.
- C. Algoritmo de Dijkstra — caminho mínimo de fonte única: a cada passo, relaxa a aresta de menor custo acumulado. Usar heap (heapq) como fila de prioridade.

Aplicar para encontrar a rota de menor custo de atendimento a partir de um hub de recursos.

Independentemente da variante escolhida, a implementação deve:

- Usar a BST para consultar municípios de alto risco a priorizar;
- Usar dicionário para manter custos acumulados e predecessores;
- Usar heap (heapq) ou conjunto para gerenciar a fronteira de expansão;
- Reconstruir e exibir o caminho/árvore ótima encontrada;
- Demonstrar a razão de escolha local a cada passo é justificável (prova informal de corretude).

4. Monitoramento de Desempenho

Para cada algoritmo e cada tamanho de instância testado ($N = 5, 8, 10, 12, 20, 50, 100$ vértices), o módulo `performance_monitor.py` deve registrar e exibir:

- Tempo de execução: em milissegundos, via `time.perf_counter()`;
- Memória alocada: em megabytes, via `tracemalloc`;
- Número de operações elementares: arestas relaxadas (Dijkstra), inserções no heap (Prim/Kruskal), chamadas recursivas (Força Bruta - FB);
- Escalabilidade empírica: curva N vs. tempo para ambos os algoritmos no mesmo gráfico.

Exigência: o relatório deve apresentar o gráfico comparativo (tempo $\times N$) para Força Bruta e Guloso e discutir a razão de o algoritmo 'Força Bruta' se torna inviável e a partir de qual 'N' isso ocorre, mesmo que empiricamente. Identifique o cruzamento das curvas e interprete o significado prático dessa atividade.

5. Casos Brasileiros

O sistema deve ser instanciado em pelo menos dois dos cenários abaixo. Os dados podem ser reais (fontes indicadas) ou sintéticos devidamente justificados. A escolha deve ser argumentada no relatório com base na disponibilidade de dados e na relevância social.

Cenário A — Rede de resposta a enchentes no Rio Grande do Sul

Grafo baseado nos 478 municípios afetados pelas enchentes do RS em 2024. Vértices = municípios; arestas = rodovias com tempo de deslocamento como peso. Objetivo: encontrar a MST de cobertura mínima para posicionar equipes de resposta, e o caminho mais curto de Porto Alegre a cada município afetado. Fonte: malha viária DNIT + dados Defesa Civil RS.

Cenário B — Triagem de risco de seca no MATOPIBA

Grafo de municípios da região MATOPIBA (MA, TO, PI, BA) com índice de risco derivado de NDVI e precipitação INMET. BST organiza os municípios por grau de criticidade. Algoritmo Guloso determina a ordem ótima de atendimento dado um orçamento máximo de deslocamento. Fonte: NDVI MODIS/NASA + INMET.

Cenário C — Cobertura de conectividade rural via satélite

Grafo de municípios sem banda larga fixa (dados ANATEL 2024). Peso das arestas = custo estimado de instalação de estação de rádio/satélite. MST determina a rede de menor custo para conectar todos os municípios ao “backbone” de internet. Fonte: ANATEL + IBGE.

Cenário D — Rotas de brigadas de fiscalização na Amazônia

Grafo de pontos de alerta DETER/INPE com arestas representando rotas fluviais e terrestres. Vértices de alto desmatamento têm maior peso de prioridade na BST. Greedy consegue encontrar a rota de menor custo de deslocamento para as brigadas a partir de bases operacionais. Fonte: INPE PRODES/DETER.

6. Avaliação Comparativa e Escala de Decisão

Não existe uma única resposta correta neste projeto. Os grupos devem classificar as soluções encontradas em uma escala de decisão com pelo menos quatro níveis, considerando o trade-off/compromisso entre qualidade da solução e custo computacional.

6.1 Figuras

Cada figura deve conter título, legenda, fonte dos dados e interpretação em texto de no mínimo três linhas:

- Visualização do grafo de municípios com arestas da MST destacadas (usando networkx + matplotlib ou similar);
- Representação visual da BST para uma instância de 10 a 15 nós (diagrama de árvore com índices de risco);
- Gráfico comparativo de desempenho: tempo de execução × N para Força Bruta e Guloso;

- Tabela de estruturas de dados: qual foi usada em cada etapa, com justificativa de complexidade;
- Gráfico de gap de otimalidade: diferença percentual entre a solução FB (ótima) e a Gulosa em função de N.

6.2 Construção da escala de decisão

A escala deve ordenar as alternativas de solução identificadas (diferentes variantes Greedy, diferentes tamanhos de instância, diferentes cenários brasileiros) considerando simultaneamente:

- Qualidade da solução: proximidade ao ótimo global (gap em relação à Força Bruta);
- Custo computacional: tempo e memória para instâncias de diferentes tamanhos;
- Adequação da estrutura de dados: a escolha impacta a eficiência? Como e por quê?
- Aplicabilidade prática: a solução é viável para o cenário brasileiro escolhido?

O relatório será avaliado pela qualidade do raciocínio apresentado, não apenas pela precisão numérica. Uma análise crítica e fundamentada do gap de otimização entre Força Bruta e Greedy reúne mais pontos do que um resultado sem interpretação.

7. Entregáveis

7.1 Repositório GitHub – acesso público

O grupo deve ter seus representantes identificados com NOME e RA.

Repositório público, com histórico de “commits” distribuído entre os integrantes, README em português com instruções de execução e descrição de cada módulo. Estrutura obrigatória:

global-solution-2026-fund/	Raiz do projeto
README.md	Descrição, instruções e dependências...identificação do grupo com RA e NOME dos integrantes.
requirements.txt	Dependências Python
data/raw/	Dados brutos: NDVI, pluviometria, malha viária
data/processed/	Grafos e árvores serializ. (JSON/pickle)
src/data_structures.py	Impl. de lista, tupla, dict, heap, árvore binária, grafo
src/brute_force.py	Enumeração completa — baseline de validação
src/greedy.py	Prim / Kruskal / Dijkstra — solução eficiente

src/performance_monitor.py	Tempo, memória, contadores de operações
src/visualizations.py	Grafos, árvores e figuras obrigatórias
notebooks/analise_resultados.ipynb	Análise interativa e escala de decisão
tests/test_algorithms.py	Testes unitários (pytest)
report/relatorio_final.pdf	Relatório ≤ 12 páginas

7.2 Relatório Técnico (PDF, máx. 4 páginas)

Seções obrigatórias, nesta ordem:

1. Identificação dos componentes do grupo (RA – NOME). Contextualização e justificativa do cenário brasileiro escolhido;
2. Modelagem: definição formal do grafo e da BST, escolha das estruturas de dados justificada;
3. Análise de complexidade teórica (tempo e espaço) dos dois algoritmos;
4. Resultados com todas as figuras obrigatórias e interpretações;
5. Escala de decisão comentada com gap de otimização e análise de desempenho;
6. Conclusão com recomendação prática e conexão com ODS;
7. Referências (fontes de dados, artigos, documentação técnica).

8. Critérios de Avaliação

Componente	Peso	Critérios principais
Compreensão e organização da solução	20%	Modelagem do grafo/árvore, escolha das estruturas de dados, README e estrutura do repositório
Código e aplicação de algoritmos	45%	Corretude de FB e Greedy, uso adequado das estruturas, testes automatizados, análise de desempenho
Discussão dos resultados e relatório	35%	Figuras obrigatórias, escala de decisão argumentada, conexão com ODS e cenário brasileiro real

9. Recursos e Fontes de Dados

9.1 Dados geoespaciais e ambientais

- NASA Earthdata — MODIS NDVI e SRTM: earthdata.nasa.gov
- INPE PRODES/DETER — desmatamento Amazônia: terrabrasilis.dpi.inpe.br

- ANA — pluviometria e hidrologia: hidroweb.ana.gov.br
- INMET — dados climáticos históricos: bdmep.inmet.gov.br
- IBGE — malha municipal e dados socioeconômicos: ibge.gov.br/geociencias
- ANATEL — cobertura de internet: mapa.anatel.gov.br
- DNIT — malha viária federal: dnit.gov.br

9.2 Referências bibliográficas sugeridas

- Cormen, T. et al. (2022). Introduction to Algorithms, 4th Ed. MIT Press. (Caps. 22–25: Grafos e Algoritmos Gulosos)
- Sedgewick, R. & Wayne, K. (2011). Algorithms, 4th Ed. Addison-Wesley. (Parte 4: Grafos)
- Skiena, S. (2020). The Algorithm Design Manual, 3rd Ed. Springer.
- Carta Internacional Space and Major Disasters: disasterscharter.org

9.3 Bibliotecas Python sugeridas

- networkx — construção e visualização de grafos (não substitui a implementação do Greedy);
- matplotlib, seaborn — geração de figuras;
- heapq — fila de prioridade nativa (obrigatório no Guloso);
- tracemalloc — monitoramento de memória;
- pytest — testes automatizados;
- geopandas, shapely — manipulação de dados geoespaciais (opcional).

A próxima corrida tecnológica já começou e participamos dela.

FIAP — Global Solution | 1º Semestre de 2026 | Dynamic Programming – Prof. André Marques